

ECEN 428/722 - Post Lab Report

Lab Number: 9

Lab Title: CNN Acceleration Using Systolic Arrays on FPGAs

Section Number: 602

Student's Name: Sai Vikhyat Pattar

Student's UIN: 936001854

Date: 12/4/2025

TA: Cristhian Roman Vicharra

Objective:

The purpose of this lab was to explore how convolutional neural networks (CNNs) can be efficiently implemented on an FPGA using systolic array architectures. The lab focused on understanding how convolution and matrix-based operations in CNNs map onto a 2D systolic array composed of multiple processing elements (PEs).

Methodology:

1. Extracted the provided Lab8_student_code.zip and lab8_base.zip files.
2. Opened the Vivado project base.xpr and added source files systolicarray_1.v, systolicarray_2.v, and systolicarray_tb.v.
3. Implemented the 3D systolic array for 2x2 matrix multiplication inside systolicarray_1.v, modeling the design shown in Figure 1 of the manual.
4. Modified the testbench to input : $\begin{bmatrix} 0 & 5 & 1 \\ 1 & 0 & 5 \end{bmatrix} \times \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$
5. Verified functionality through behavioral simulation in Vivado and captured the result waveform.
6. Designed the 2D systolic array as described in Figure 2 of the manual and implemented it in systolicarray_2.v.
7. Reused and modified the testbench to verify the 2D design through simulation.
8. Synthesized and implemented both versions in Vivado, ensuring proper timing closure.
9. Added a True Dual-Port RAM (8-bit width, 65536-depth) to top_systolicarray.v for FPGA integration.
10. Generated the bitstream, exported the design to Vitis, created BOOT.bin, and executed ./lab8_sysarr_test on the Zybo board.
11. Captured screenshots of terminal output, hardware utilization, power, timing, and throughput reports.

Design:

The design was composed of several Verilog modules, each responsible for a different section of the accelerator:

- **top_sysarr.v** – Acts as the top-level system wrapper, linking the CNN accelerator with memory while coordinating the execution of CNN layers.
- **matrix_cal.v** – Converts CNN layer parameters into appropriate memory transactions and control signals required to drive the systolic array.
- **systolic_array.v** – High-level structural module that connects buffers and the PE array to create a functioning systolic compute fabric.
- **array.v** – Builds the multi-dimensional PE grid and establishes communication between PEs.

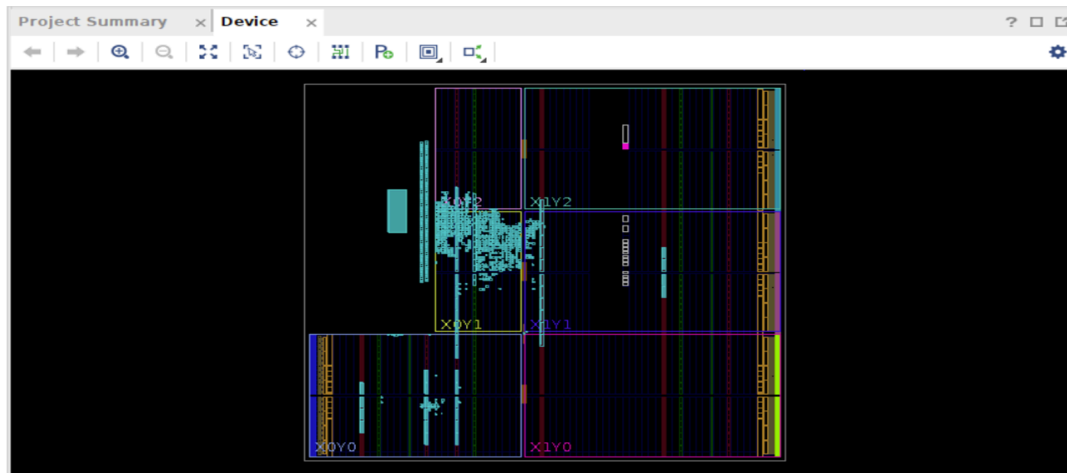
- **PE.v** – Implements the core processing element, which performs repeated MAC operations essential to CNN computation.
- **multiply.v** – Provides a fixed-point multiplication unit used within PEs for weight-activation multiplication.
- **IB.v** – Input/weight buffering module that streams data efficiently into the array while maximizing reuse.
- **OB.v** – Collects partial sums from the systolic array and writes them back to memory under supervision of the controller.

Implementation:

- Cloned the original Vivado project and added the Verilog modules for **PE**, **array**, and **systolic_array**, designating **top_sysarr.v** as the project's top-level file.
- Generated a **true dual-port block RAM** IP core (**blk_mem_gen_0**) for **top_sysarr.v**, configured with an 8-bit data width, 65,536-word depth, read-first behavior, and permanently enabled ports.
- Generated a second memory IP (**blk_mem_gen_1**) as a **single-port block RAM**, set to 16-bit width, 65,536-word depth, read-first mode, and always-enabled access.
- Completed synthesis, implementation, and bitstream creation in Vivado, then exported the hardware design along with the compiled bitstream.
- Imported the exported hardware into SDK/Vitis, built the corresponding software platform, and created the Zybo boot image.
- Transferred **BOOT.bin**, Linux kernel files, the device tree, **lab9_cnn_test**, and the CNN dataset onto the SD card, then configured the board to boot via SD.
- Powered up the Zybo board, launched Linux, executed **./lab9_cnn_test** to perform MNIST inference on the accelerator, and logged the FPGA cycle counts.
- Gathered utilization, timing, power consumption, maximum achievable clock frequency, and throughput figures from Vivado's post-implementation reports and runtime measurements.

Results:

Implemented Design and Timing Analysis:



Terminal output:

```
zynq> cd /mnt                                     # Change directory to /mnt [web:1]
zynq> insmod transfpga.ko                         # Insert the kernel module transfpga
.ko [web:1]
Mapping virtual address...
The physical address is 0x43c00000
The virtual address is 0x608e0000
Registered a device with dynamic Major number of 245
Create a device file for this device with this command:
'mknod /dev/transfpga c 245 0'.
zynq> mknod /dev/transfpga c 245 0 # Create a character device node fo
r transfpga with major 245 and minor 0 [web:1]
zynq> ./lab9_cnn_test                             # Run the lab9_cnn_test program [web
:1]
send the figure and weights to FPGA
trigger systolic array
wait for systolic array
read back the results
testcase 0, probability: 0->-5.882812
testcase 0, probability: 1->-4.394531
testcase 0, probability: 2->2.496094
testcase 0, probability: 3->4.058594
testcase 0, probability: 4->-8.437500
testcase 0, probability: 5->-1.867188
testcase 0, probability: 6->-20.035156
testcase 0, probability: 7->11.863281
testcase 0, probability: 8->0.011719
testcase 0, probability: 9->-0.183594
testcase 0, the number is 7
correct
send the figure and weights to FPGA
trigger systolic array
wait for systolic array
read back the results
testcase 1, probability: 0->-12.523438
testcase 1, probability: 1->-1.234375
testcase 1, probability: 2->15.214844
testcase 1, probability: 3->-4.339844
testcase 1, probability: 4->-37.539062
testcase 1, probability: 5->-21.917969
testcase 1, probability: 6->-28.324219
testcase 1, probability: 7->-8.667969
testcase 1, probability: 8->-7.429688
testcase 1, probability: 9->-34.175781
testcase 1, the number is 2
correct
accuracy: 0.956000
```

Slack table:

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS): 26.428 ns		Worst Hold Slack (WHS): 0.023 ns		Worst Pulse Width Slack (WPWS): 15.520 ns	
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns		Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	
Total Number of Endpoints: 1687		Total Number of Endpoints: 1687		Total Number of Endpoints: 668	
All user specified timing constraints are met.					

Power consumption :

Summary	
Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.	
Total On-Chip Power:	1.564 W
Design Power Budget:	Not Specified
Process:	typical
Power Budget Margin:	N/A
Junction Temperature:	43.0°C
Thermal Margin:	42.0°C (3.5 W)
Ambient Temperature:	25.0 °C
Effective θJA:	11.5°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	Low
Launch Power Constraint Advisor to find and fix invalid switching activity	
On-Chip Power 91% 9% Dynamic: 1.422 W (91%) Device Static: 0.142 W (9%) Clocks: 0.001 W (<1%) Signals: 0.002 W (<1%) Logic: 0.001 W (<1%) BRAM: 0.017 W (1%) DSP: 0.002 W (<1%) PS7: 1.399 W (95%)	

Hardware Utilization :

1. Slice Logic

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs*	1586	0	0	53200	2.98
LUT as Logic	1516	0	0	53200	2.85
LUT as Memory	70	0	0	17400	0.40
LUT as Distributed RAM	0	0			
LUT as Shift Register	70	0			
Slice Registers	1100	0	0	106400	1.03
Register as Flip Flop	1100	0	0	106400	1.03
Register as Latch	0	0	0	106400	0.00
F7 Muxes	18	0	0	26600	0.07
F8 Muxes	9	0	0	13300	0.07

* Warning! The Final LUT count, after physical optimizations and full implementation. Warning! LUT value is adjusted to account for LUT combining.

Utilization		Post-Synthesis Post-Implementation		
		Graph Table		
Resource	Utilization	Available	Utilization %	
LUT	1448	53200	2.72	
LUTRAM	60	17400	0.34	
FF	1088	106400	1.02	
BRAM	46	140	32.86	
DSP	15	220	6.82	
BUFG	2	32	6.25	

Maximum clock frequency

The maximum clock frequency is calculated using:

$$\begin{aligned}
 f_{\max} &= 1 / (\text{Total clock period} - \text{Worst Negative Slack}) \\
 &= 1 / (33 - 26.428) \\
 &\approx 152.16 \text{ MHz}
 \end{aligned}$$

Since behavioral simulation results are not available, the actual throughput cannot be directly measured. However, we can still estimate the theoretical maximum throughput.

The systolic array used in the design contains 64 processing elements arranged in an 8×8 grid. Each PE performs one MAC operation per clock cycle when active.

Therefore, the theoretical peak throughput at the maximum clock frequency is:

$$\begin{aligned}
 \text{Throughput} &= \text{Number of PEs} \times f_{\max} \\
 &= 64 \times 152.16 \text{ MHz} \\
 &\approx 9.74 \text{ GMAC/s}
 \end{aligned}$$

Final result: The systolic array can theoretically achieve about 9.74 GMAC/s of peak compute throughput.

Conclusion:

This lab provided practical experience in implementing and evaluating a systolic-array-based CNN accelerator on an FPGA. By integrating PE, array, and memory modules into a complete hardware system and deploying it on the Zybo board, I gained a deeper understanding of dataflow architectures, fixed-point computation, and hardware–software co-design. The timing, power, and utilization results validated that the design meets implementation constraints, and the theoretical performance analysis demonstrated the high computational efficiency achievable with systolic arrays. Overall, the lab strengthened my skills in RTL design, FPGA toolflow, and hardware acceleration for machine learning applications.